

Embedded Systems Architectures - Laboratory Guide 5

Software Device Drivers

Academic Year 2025/2026

Authors: Arnaldo Oliveira, Fábio Coutinho

1. Objectives

- Analysis and test of a device driver source code for the I2C TC74 temperature sensor
- Development and test of a device driver for the I2C DHT20 temperature and humidity multi-sensor
- Utilization of the oscilloscope to validate serial interface transfers with focus on I2C

2. Introduction

Software device drivers are a fundamental part of embedded systems because they provide the interface between application code and hardware peripherals or external devices. In the context of microcontrollers, a device driver written in C for a simple sensor/actuator is typically a compact software module that allows a bare-metal application to configure the sensor/actuator, read its measurements, write its state and sometimes perform basic control operations such as calibration, reset, or mode selection.

A bare-metal application is one that runs directly on the microcontroller without an operating system. In this environment, the programmer has full control of the hardware, but also full responsibility for initialization, timing, communication, and error handling. As a result, device drivers for bare-metal systems are usually

designed to be small, deterministic, and efficient, with minimal memory and processing overhead.

Simple sensors connected to microcontrollers may include temperature, humidity, light, pressure, accelerometers, or proximity detectors. These sensors are commonly interfaced through protocols such as GPIO, ADC, I2C, SPI or RS232. A device driver hides the low-level communication details of these interfaces and exposes a simpler set of functions to the application. For example, instead of forcing the application to manually generate I2C transactions and interpret register values, the driver may offer functions such as *sensor_init()*, *sensor_read_temperature()*, or *sensor_set_mode()*.

In C, such drivers are often organized into a header file and a source file. The header file defines the interface, i.e., public data types, constants, and externally visible function prototypes, while the source file contains the implementation. Good driver design promotes modularity and portability. This means that hardware-specific operations, such as reading or writing bytes on an I2C bus, should be separated as much as possible from sensor-specific logic. This approach makes the driver easier to reuse across different microcontroller families.

A typical driver workflow includes sensor initialization, optional configuration, data acquisition, and data conversion. Initialization may involve setting up communication peripherals and verifying that the sensor is present. Configuration may include selecting measurement resolution, sampling frequency, or operating mode. Data acquisition usually consists of reading one or more sensor registers, while data conversion transforms raw digital values into meaningful physical units such as degrees Celsius or lux.

Because bare-metal systems do not rely on operating system services, timing is often handled through polling loops, hardware timers, or interrupt routines. Therefore, the driver must be designed with attention to execution time, blocking behavior, and reliability. For simple applications, polling-based drivers are common and easy to

understand. In more time-sensitive systems, interrupt-driven approaches may be preferable.

Overall, software device drivers in C are essential for structuring embedded applications that interact with sensors. They improve code readability, maintainability, and reuse by separating hardware access from application logic. In bare-metal microcontroller systems, well-designed drivers are especially important because they help manage complexity while preserving the efficiency and predictability required in resource-constrained environments.

In this class, we will analyse, develop and test two device drivers for the TC74 and DHT20 sensors.

3. Practical Work

The practical work in this class is based on the previous lab handout, the technical data sheets for the TC74 and DHT20 sensors and the Reference API for the ESP32-C6 peripherals and the ESP32-C6 Technical Reference Manual: <https://documentation.espressif.com/esp32-c6 technical reference manual en.pdf>

Warning! This lab handout assumes that all previous tasks were completed successfully. Besides that, it can help resolve issues from the previous I2C lab.

3.1 Analysis and test of a device driver source code for the TC74 temperature sensor

Analyse the source code provided in the *temp_sensor_tc74.h* and *temp_sensor_tc74.c* files available on the course website.

Question 1: Define a strategy for a step-by-step test and validation of the functions provided in the device driver, with the help of an adequate `main()` function and a two-channel oscilloscope, to allow the visualization of the transfer performed by each function, which must be individually called in a periodic way (inside an infinite loop with an adequate delay).

Question 2: Why should the `printf()` function or other functions that introduce considerable delay or jitter not be called in loops used to generate signals with precise timing and/or to be shown in the oscilloscope?

Question 3: Which device driver function(s) cannot be tested with the help of the oscilloscope?

Question 4: What is a possible test order of the device driver functions?

Question 5: For the visualization of an I2C transfer, consisting of the SDA and SCL signals, which signal must be used as the trigger of the oscilloscope? What should be the slope/edge and trigger level used? Hint: recall the timing diagram of an I2C start condition.

Question 6: Create a new project workspace with the device driver header and implementation source files and write a `main()` function (in a separate source file, e.g., `tc74_test.c`) that will evolve to implement the test strategy defined in the previous questions.

Question 7: After performing the tests of the device driver, write a final `main()` function for initialization of the device driver, reading the temperature, and outputting the value in the terminal.

3.2 Development and test of a device driver for the DHT20 multi-sensor

Define the interface (.h file) and the implementation (.c file) for a device driver for the DHT20 multisensor, following a code organization, a step-by-step testing approach, and a user application (`main()` function in a separate .c file) similar to the one used for the TC74 sensor, but outputting the temperature and the humidity provided by the DHT20. The workspace should include three “C” source files:

- Device driver header file (e.g., `multisensor_dht20.h`)
- Device driver implementation file (e.g., `multisensor_dht20.c`)
- Application implementation file (e.g., `dht20_test.c`)